

Inno3D HBM - Feedback Assessment

Feedback Items 1-8, Roadmap Items, and Technical Discussion Points

Updated Assessment and Proposed Plan

Date: 16 July 2026

Purpose

This document reviews the customer feedback item by item and provides an updated assessment covering items that can be completed within the current plan, existing features that require refinement, new roadmap features requiring additional effort, estimated working days, proposed priorities, and technical recommendations.

Important Scope Note

Item 6 - Additional Features to Implement is classified as Phase-2 / roadmap scope. These advanced features will be documented in detail but will not affect the existing committed project plan or current delivery schedule.

1. Executive Summary

Category	Coverage	Assessment
Current plan / refinement items	Items 1, 2, 3, 4, 7, 8 and layout refinement from Item 5	Estimated scope excluding fullscreen clarification: approximately 14 WD.
Clarification required	Fullscreen behavior	Estimate depends on the finalized fullscreen behavior: current in-grid view maximize rename, application-level fullscreen, or true monitor-level fullscreen.
Roadmap / Phase-2	Item 6 - advanced visualization and measurement tools	22 WD. This is a separate roadmap package and is not included in the current plan.
Technical discussion points	Dynamic algorithm parameters and deep learning model deployment	Design recommendations only in this document. No time estimate is included unless implementation planning is requested separately.

2. Assessment Principles

- The Python Inno3D prototype is considered a functional reference only. The WPF/VTK application can be redesigned where needed for better usability, logic, performance, and maintainability.
- A major UI redesign is not recommended at this stage because planned implementation tasks are already in queue. UI professionalization should be handled incrementally together with related tasks.

- Advanced visualization and measurement features requested under Item 6 are moved to the end of the plan as Phase-2 / roadmap items.
- Working-day estimates include implementation and basic regression testing, subject to final customer confirmation and dataset validation.
- Any item marked as clarification required should not be committed until the expected behavior is confirmed.

3. Proposed Priority Classification

Priority	Meaning	Typical Items
P1	Important corrections and usability refinements	Bug fixes, Reverse Z consistency, pixel value format, button states, 3D crosshair visibility.
P2	Existing feature refinements and input support	Layout selector refinement, RAW support, drag-and-drop, default 3D settings, sidebar polish.
P3	Roadmap / Phase-2 enhancements	Interactive Oblique MPR, 3D alignment, thick slab, crop box, measurement tools.
Technical recommendation	Architecture discussion only	JSON-based dynamic parameter configuration and wrapper-based deep learning deployment.

4. Detailed Feedback Assessment

Item 1 - UI/UX

Feedback	Classification	Recommended Action	Estimate	Priority
Button hierarchy and iconography	Existing UI refinement	Introduce clear primary/secondary/icon button sizing and improve unclear icons such as crosshair, color selection, mask visibility, fullscreen/restore, Reverse Z, layout, LUT and reset.	1 WD	P1
Button state color contrast	Existing UI refinement	Define consistent normal, hover, pressed, active, selected and disabled states across Viewer, Teaching and floating toolbar controls.	1 WD	P1
Inno3D logo and HBM subtitle	Minor UI polish	Use Casanova Scotia only if customer provides or approves the licensed font. Otherwise, refine the logo text style and add the detail line: High Bandwidth Memory (HBM).	Included	P2
Window minimize/maximize visual distinction	Existing UI correction	Use clearly different minimize, maximize and restore symbols with consistent button sizing and hover state.	Included	P1
Fullscreen mode	Clarification required	The expected fullscreen behavior needs to be finalized: current in-grid view maximize, application-level fullscreen, or true monitor-level fullscreen. If the current behavior is intended only for per-view enlargement, it can be renamed as	TBD	Clarify

		Maximize View for better clarity.		
Borderless layout and grid-layout icons	Existing feature refinement	Improve the current layout presentation with a cleaner borderless/semi-borderless look and add predefined layout icons such as 2x2, Large 3D, Large XY, vertical split, horizontal split and selected-view maximize. Reuse existing layout/splitter logic and preserve loaded data, masks, camera and slice state while switching layouts.	3 WD	P2
Sidebar width	Minor UI polish	Reduce startup left sidebar width and improve spacing without changing the existing workflow.	Included	P2
Default 3D settings	Existing configuration update	Set default projection to Perspective, LUT to Grayscale, and initial rendering quality/opacity/sharpness to the customer-preferred values while keeping other defaults stable.	0.5 WD	P2

UI Professionalization Strategy

A full UI redesign should not be introduced in the current phase because it may affect the existing plan and create regression risk. Instead, UI improvements should be applied incrementally while working on related tasks: button style during button-state work, layout icons during layout switching, status/pixel information during pixel-value work, scale bar during ruler work, and control cleanup during related viewer/teaching updates.

Item 2 - Crosshair | Ruler

Feedback	Classification	Recommended Action	Estimate	Priority
Replace vertical ruler with horizontal scale bar	Existing feature refinement	Use a scientific horizontal scale bar similar to Zeiss-style reports. The scale bar should be placed inside XY/XZ/YZ views, update with zoom, and show physical units using voxel spacing.	2 WD	P1
3D volume crosshair too small / hard to locate	Existing feature refinement	Increase 3D crosshair visibility with larger lines, clearer center marker and better contrast while keeping synchronization with 2D crosshair and current slice position.	1 WD	P1

Item 3 - Pixel Value

Feedback	Classification	Recommended Action	Estimate	Priority
Coordinate origin / axis direction indicator	Existing feature refinement	Add compact axis-direction indicator in	1 WD	P1

missing		each viewport. Suggested placement is bottom-right because the top-right area is already used by the floating toolbar. Indicators should reflect XY, XZ and YZ orientation and update with Reverse Z.		
Pixel value display format	Existing feature refinement	Use the format Pixel (x,y,z): value. Coordinates should be calculated based on current view orientation, current slice, crosshair position and Reverse Z state.	Included in above	P1

Item 4 - Reverse Z

Feedback	Classification	Recommended Action	Estimate	Priority
Reverse Z should apply to full volume and coordinate system	Existing feature correction/refinement	Apply Reverse Z consistently to XY/XZ/YZ displayed views, slice-index mapping, crosshair synchronization, mask overlay alignment, pixel coordinate display and axis-direction indicator.	1 WD	P1

Reverse Z Scope Boundary

This estimate covers display and coordinate consistency in the viewer. It does not include deep inspection-pipeline coordinate remapping, object-statistics remapping, saved-state migration, or future measurement-tool integration.

Item 5 - Grid Layout Switching

Feedback	Classification	Recommended Action	Estimate	Priority
Current viewer supports only fixed 2x2 layout	Existing feature refinement	Covered under Item 1 - Borderless layout and grid-layout icons. Provide predefined layout options and visually clear layout icons while reusing existing splitter/layout logic.	Covered in Item 1	P2

Item 6 - Additional Features to Implement

Roadmap / Phase-2 Classification

The following features are advanced visualization and measurement capabilities. They should be documented as Phase-2 / roadmap items and moved to the end of the existing project plan. They will not affect the current committed scope or delivery schedule.

Roadmap Item	Initial Scope / Detail	Estimate	Priority
--------------	------------------------	----------	----------

Interactive Oblique MPR	Add a basic interactive oblique cutting plane to inspect the volume in a non-orthogonal direction. User can adjust or rotate the plane and view the generated oblique slice. A reset option will be provided. The implementation will reuse existing LUT, W/L, zoom/pan and rendering controls where possible. Advanced multi-plane oblique layouts and measurement integration can be considered later.	6 WD	P3
3D Volume Alignment	Provide basic manual alignment controls for the loaded 3D volume. User can apply simple translate/rotate adjustment and reset back to the original alignment. This is mainly for display/orientation correction. Automatic registration, multi-volume alignment and AI-based alignment are excluded from the first scope.	3 WD	P3
Thick Slab	Add slab-thickness control for MPR views so multiple neighbouring slices can be combined into one displayed image. Initial modes will include Normal, Thick MPR/Average and MAXIP/MIP. Shaded mode will be reviewed based on the exact visual expectation from the customer.	4 WD	P3
3D Cropping Box / Interactive Bounding Box	Add an interactive crop box in the 3D volume view by reusing the existing clipping logic. User can adjust the crop region visually and restore the full volume using a Reset Clip Box button. Advanced saved crop presets or complex ROI management are excluded from the first scope.	3 WD	P3
Measurement Tools	Add a basic measurement toolset covering 2D distance, 2D/3D angle, 2D area, 3D volume and line profile. Measurements will use voxel spacing where applicable and display results in appropriate units. Basic clear/delete and result display will be included. Advanced measurement history, export, annotation management and oblique-MPR measurement integration can be considered later.	6 WD	P3

Phase-2 Roadmap Total	Estimate
Item 6 total estimated effort	22 WD

Roadmap Estimate Note

The above roadmap estimates are based on a lean first implementation. The goal is to provide practical usable versions of the requested advanced features without changing the current committed project plan. More advanced behavior can be expanded later based on customer priority and feedback.

Item 7 - Input File Support

Feedback	Classification	Recommended Action	Estimate	Priority
Drag-and-drop files/folders	Existing input workflow enhancement	Allow dropping a valid volume file or image folder into the load area and reuse existing load/validation logic. Unsupported paths should show a clear error message.	0.5 WD	P2
Support single .raw volume file	New input format support	Add raw file loading with required metadata such as width, height, depth, bit depth/data type, byte order, header size and slice order.	1 WD	P2
Support folder containing .raw sequence files	New input format support	Load raw slice sequence folder with correct ordering and required metadata. Reuse existing folder-loading pipeline where possible.	1 WD	P2

RAW Metadata Requirement

RAW files normally do not contain internal metadata. The required volume metadata should be supplied through a companion config file, user input, or a agreed naming/metadata convention.

Item 8 - Bug Fixes and Rendering Issues

Feedback	Classification	Recommended Action	Estimate	Priority
GPU VRAM display includes system/shared memory	Bug fix	Display dedicated GPU VRAM only and exclude shared GPU/system RAM. Validate against Windows Task Manager dedicated GPU memory.	0.5 WD	P1
Binary volume visualization shows solid gray block	Bug fix	Detect 8-bit binary volume/mask with values 0 and 255 and render as binary mask-style visualization: 0 as transparent/background and 255 as visible.	0.5 WD	P1

5. Estimated Effort Summary

Area	Estimate	Notes
Item 1 - UI/UX refinements	Approx. 5.5 WD	Excludes fullscreen clarification item. Borderless layout/grid-layout icons are limited to 3 WD by reusing existing layout logic.
Item 2 - Crosshair Ruler	3 WD	Horizontal scale bar and focused 3D crosshair visibility improvement.
Item 3 - Pixel Value	1 WD	Coordinate indicator and pixel format.

Item 4 - Reverse Z	1 WD	Display and coordinate consistency scope.
Item 5 - Grid Layout Switching	Covered in Item 1	No separate count.
Item 7 - Input File Support	2.5 WD	Drag-and-drop, single .raw and raw sequence folder support.
Item 8 - Bug Fixes	1 WD	GPU VRAM display and binary volume visualization.
Current plan / refinement subtotal	Approx. 14 WD	Excludes fullscreen clarification and excludes Item 6 Phase-2 roadmap.
Item 6 - Additional Features	22 WD	Phase-2 / roadmap only. Not part of current committed plan.

6. Technical Discussion Points

6.1 Dynamic Algorithm Parameters

Recommendation: Use a JSON-based metadata-driven parameter system. Instead of hardcoding every algorithm parameter directly in the WPF UI, the application can read a structured JSON parameter definition and dynamically generate the required controls. This approach keeps the UI and algorithm configuration loosely coupled and reduces repeated UI-side code changes when parameters are added, removed, renamed, grouped or modified.

- The JSON file should define parameter name, display name, group/category, data type, default value, min/max range, unit, description/help text, visibility, basic/advanced flag, UI control type and validation rules.
- Parameter groups can be organized logically such as Bump Parameters, Void Parameters, Layer Parameters, Deep Learning Enhancement, GPU Settings and Output Settings.
- The same metadata can drive UI generation, validation, default/reset behavior, save/reload behavior and tooltip/help text.
- This reduces mismatch risk between algorithm-side parameters and UI-side controls and makes future algorithm updates easier to deploy.

Example JSON definition:

```
{
  "group": "Void Parameters",
  "parameters": [
    {
      "name": "VOID_MIN_AREA",
      "displayName": "Void Min Area",
      "type": "double",
      "default": 15.0,
      "min": 0.0,
      "max": 5000.0,
      "unit": "um2",
      "control": "numeric",
      "description": "Minimum void area used for filtering small noise regions.",
      "visible": true,
      "advanced": false
    },
  ],
}
```

```
"name": "USE_GPU",
"displayName": "Use GPU Acceleration",
"type": "bool",
"default": true,
"control": "checkbox",
"description": "Enable GPU processing for supported algorithm steps.",
"visible": true,
"advanced": false
}
]
}
```

Parameter Metadata	Purpose
name	Internal key passed to the algorithm/config file.
displayName	User-friendly label shown in the UI.
group	Logical UI grouping such as Bump, Void, Layer, DL Enhancement or GPU.
type	Defines how the value is parsed and validated: int, double, bool, enum, path, etc.
default / min / max / unit	Supports reset, validation and display in correct engineering units.
control	Defines generated UI control such as numeric input, checkbox, combo box, file browser or color picker.
description	Used for tooltip/help text and improves operator understanding.
visible / advanced	Supports basic vs advanced parameter views without code changes.

Parameter Type	Generated UI Control
int / double	Numeric input / spin control
bool	Checkbox / toggle
string	Text box
enum	Combo box
filePath	Text box + browse file button
folderPath	Text box + browse folder button
color	Color picker
range	Min/max input or slider

Technical Boundary

This approach works well for normal algorithm parameters such as thresholds, numeric values, flags, paths and enum options. If a new parameter requires a special visualization, custom workflow, preview window, graph, or algorithm-specific interaction, additional UI-side implementation may still be required. However, for most standard parameters, the JSON-driven approach provides a scalable and maintainable solution.

6.2 Deep Learning Model Deployment

Recommendation: Use a dedicated DLL/API wrapper for inference rather than executing Python scripts directly from the WPF application. Python scripts are useful during R&D and model validation, but for production integration, a wrapper-based approach provides better performance control, cleaner deployment, stable application integration and improved maintainability.

Approach	Assessment
Direct Python script execution	Useful for R&D, model validation and prototype testing. Not recommended as the final production integration approach because deployment, dependency control and error handling become harder.
DLL/API inference wrapper	Recommended for production integration. Provides a stable interface between the WPF application and the model, improves performance control, simplifies deployment and keeps model details isolated from the main application.
Practical transition approach	Continue using scripts during model development and validation. Move to wrapper-based deployment for final application integration.

Suggested wrapper interface:

```
Initialize(modelConfigPath, gpuDeviceId)
EnhanceVolume(inputPath, outputPath, progressCallback)
GetStatus()
Cancel()
Release()
```

- The WPF application should call stable wrapper methods such as Initialize, EnhanceVolume, GetStatus, Cancel and Release.
- Model-specific details such as preprocessing, patch size, overlap, batch size, runtime selection, GPU device ID, TensorRT cache path and post-processing should be handled through a separate model configuration file.
- For large HBM volume data, the inference layer should support chunk/tile-based processing, GPU memory validation, progress reporting, cancellation, model-version checking and clear error/status reporting.
- This keeps the main application independent from frequent model-side changes and makes future model updates easier to deploy.

Example model configuration:

```
{
  "modelName": "HBM_Enhancement",
  "modelVersion": "1.0.0",
  "runtime": "TensorRT",
  "inputType": "uint8",
  "outputType": "uint8",
  "patchSize": [512, 512, 64],
```

```

"overlap": [32, 32, 8],
"batchSize": 2,
"normalization": "0_1",
"useGPU": true,
"gpuDeviceId": 0,
"trtCachePath": "./cache/enhancement.trt"
}

```

Deployment Recommendation

Python scripts can continue to be used during model R&D and testing. For the final application integration, the recommended direction is wrapper-based deployment using a stable DLL/API boundary.

7. Open Clarifications

Topic	Clarification Required
Fullscreen mode	Finalize the expected fullscreen behavior: current in-grid view maximize, application-level fullscreen, or true monitor-level fullscreen.
Shaded Thick Slab	Clarify expected visual behavior for shaded slab mode before implementation.
RAW metadata	Confirm how RAW dimensions, bit depth, byte order, header size and slice ordering should be provided.
3D Volume Alignment	Current assumption is manual alignment only. Automatic registration is excluded from the first roadmap scope.
Measurement tools	Initial roadmap scope is basic measurement. Advanced measurement history, export, annotation management and oblique-MPR measurement integration are excluded.
DL wrapper boundary	Confirm final wrapper API, model package format, runtime selection and GPU/TensorRT deployment details.

8. Proposed Next Steps

- Confirm clarification items, especially fullscreen behavior, RAW metadata and deep learning wrapper boundary.
- Proceed with current-plan corrections and refinements without adding Item 6 Phase-2 features into the active delivery schedule.
- Document Item 6 as a Phase-2 roadmap package and discuss implementation priority after the current committed scope is completed.
- Use the technical discussion points as design recommendations for long-term maintainability and integration planning.
- Continue incremental UI professionalization together with related functional tasks instead of introducing a full UI redesign in the current phase.